

Coding & AWS You Need Before Building Agents

The prerequisite toolkit: the Python, LLM, and AWS skills an experienced engineer should have *before* writing agentic systems — with account bootstrap from zero.

Prepared for Kaniz

Companion to Guide 2 (Enterprise Agentic) and Guide 3 (Enterprise MCP).

How to read this guide

Read top to bottom once, then use it as a checklist. Blue = key idea, green = best practice, orange = pitfall, dark = runnable code / CLI. Every section ends with a readiness line so you know when you can tick it off.

Contents

Part A · Python foundations for agents

- A1 Project setup, venvs & dependencies
- A2 Type hints & Pydantic (tool schemas)
- A3 Async/await & concurrency
- A4 HTTP, retries & timeouts
- A5 JSON & JSON Schema
- A6 Logging, errors & testing
- A7 Decorators & context managers

Part B · LLM & API fundamentals

- B1 Tokens, context, cost & temperature
- B2 Messages, system prompts & tool calling
- B3 Streaming & structured output
- B4 Embeddings, vectors & RAG
- B5 Prompt engineering & evals basics

Part C · AWS account & security from zero

- C1 Account bootstrap & root hygiene
- C2 IAM Identity Center, CLI & profiles
- C3 IAM deep: policies, roles, least privilege
- C4 Secrets, KMS, budgets & tagging

Part D · Core AWS services for agents

- D1 S3 · D2 Lambda · D3 API Gateway
- D4 DynamoDB · D5 Step Functions
- D6 CloudWatch, X-Ray & OpenTelemetry
- D7 VPC basics · D8 ECR & Docker

Part E · Bedrock setup & IaC

- E1 Enable Bedrock & first Converse call
- E2 Knowledge Bases & Guardrails
- E3 IaC: CloudFormation / CDK / Terraform

Part F · Readiness checklist

Part A · Python foundations for agents

You know Python. This part is the *subset* that agentic code leans on hardest, where weak spots cause the most pain. If you can do all of A, the SDKs (Bedrock, Strands, FastMCP) will feel obvious.

A1 Project setup, venvs & dependencies

Agentic projects pull in many libraries (SDKs, frameworks, vector clients) and must reproduce exactly in a container later. Isolate per project and pin versions.

```
python3 -m venv .venv && source .venv/bin/activate # or: uv venv
pip install boto3 strands-agents mcp pydantic httpx tenacity python-dotenv
pip freeze > requirements.txt # exact pins for the container
```

Prefer a *pyproject.toml* for real projects; keep secrets out of code with a local *.env* (git-ignored) loaded via *python-dotenv*. Never commit credentials.

Readiness

You can create an isolated env, install + pin deps, and load config from env vars — not hard-coded.

A2 Type hints & Pydantic (tool schemas)

This is the single most useful skill for tool building. A tool's input **schema** is what the model reads to decide how to call it. Frameworks derive that schema from your type hints / Pydantic models, so good typing = good tool calling.

```
from pydantic import BaseModel, Field
from typing import Literal

class DisputeRequest(BaseModel):
    account_id: str = Field(..., description='12-digit account number')
    amount: float = Field(..., gt=0, description='Disputed amount in USD')
    reason: Literal['fraud', 'duplicate', 'not_received']

# Pydantic gives you validation + a JSON Schema for free:
print(DisputeRequest.model_json_schema())
```

Constrained types (*Literal*, *gt=0*, required fields) stop the model from passing garbage and make failures explicit. Treat tool signatures as part of your prompt engineering.

Readiness

You can model an input with Pydantic, add field descriptions/constraints, and emit its JSON Schema.

A3 Async/await & concurrency

MCP and many agent frameworks are async. Agents also call several tools at once, so understanding *asyncio* matters for latency and cost.

```
import asyncio, httpx

async def fetch(client, url):
    r = await client.get(url, timeout=10)
    return r.json()

async def main(urls):
    async with httpx.AsyncClient() as client:
        # run tool calls concurrently instead of one-by-one
        return await asyncio.gather(*(fetch(client, u) for u in urls))

asyncio.run(main(...))
```

Pitfall

Blocking calls (plain *requests*, heavy CPU) inside an async loop freeze everything. Use async clients, or offload blocking work with *asyncio.to_thread*.

Readiness

You understand the event loop, can await coroutines, and run independent calls concurrently with *gather*.

A4 HTTP, retries & timeouts

Tools call external APIs; networks fail. Every external call needs a timeout and a bounded retry with exponential backoff — otherwise one flaky API hangs or loops the whole agent.

```
import httpx
from tenacity import retry, wait_exponential, stop_after_attempt, retry_if_exception_type

@retry(wait=wait_exponential(min=1, max=20), stop=stop_after_attempt(4),
       retry=retry_if_exception_type(httpx.HTTPError))
def get_balance(account_id: str) -> dict:
    r = httpx.get(f'https://core-bank/accounts/{account_id}', timeout=8)
    r.raise_for_status()
    return r.json()
```

Readiness

Every outbound call you write has an explicit timeout and a capped, backed-off retry.

A5 JSON & JSON Schema

Tool inputs/outputs, model responses, and MCP messages are all JSON. You should be fluent reading a JSON Schema (types, required, enum, nested objects) because that's the contract the model sees. Keep tool **outputs** small and structured — every returned token re-enters the context and costs money.

Readiness

You can read/write a JSON Schema by hand and design compact, predictable tool outputs.

A6 Logging, errors & testing

Agents are non-deterministic; observability starts in your code. Use structured logging (key/value), return errors the model can reason about rather than throwing raw stack traces into context, and unit-test tools with mocked external calls.

```
import logging, json
log = logging.getLogger('agent')

def safe_tool(fn):
    def wrap(**kw):
        try:
            return fn(**kw)
        except Exception as e:
            log.warning('tool_error', extra={'tool': fn.__name__, 'err': str(e)})
            return {'error': type(e).__name__, 'detail': str(e)} # model can react
    return wrap
```

Readiness

You log structured events, convert exceptions into model-readable errors, and can pytest a tool with the network mocked.

A7 Decorators & context managers

You don't need to be an expert, but FastMCP (`@mcp.tool()`), Strands (`@tool`), and AgentCore (`@app.entrypoint`) are all decorators, and clients/sessions are context managers (*with*). Recognising the pattern makes every SDK readable at a glance.

Readiness

You can read decorator-based SDK code and use *with* for sessions/clients without surprise.

Part B · LLM & API fundamentals

Agents are LLMs in a loop. These are the model-level concepts that drive correctness, latency and cost.

B1 Tokens, context, cost & temperature

- **Token** — a sub-word chunk; models read and bill per token (input + output). ~4 chars ≈ 1 token in English.
- **Context window** — the max tokens a single call can hold (prompt + tools + history + output). Agents fill this fast; prune aggressively.
- **Cost** — roughly (input tokens × in-rate) + (output tokens × out-rate), per model call. An agent makes *many* calls per task, so loop length and tool-output size dominate the bill.
- **Temperature** — randomness. Use low (0–0.3) for tool-using/agentive work where you want consistent decisions; higher only for creative text.

The agent cost mental model

Cost ≈ (steps in the loop) × (tokens carried each step). Two levers: cap iterations, and keep tool outputs + history small. This is why we cap loops and trim context in Guides 2–3.

B2 Messages, system prompts & tool calling

Modern APIs use a messages list of roles (system, user, assistant, tool). The **system prompt** sets behaviour and rules. **Tool calling** is the model returning a structured request to invoke a function (name + JSON args) instead of plain text — the mechanism the whole agent loop is built on (you saw the Bedrock Converse shape in the first field guide).

Readiness

You can explain the roles, write a tight system prompt, and trace one tool-call round-trip end to end.

B3 Streaming & structured output

Streaming yields tokens as they're produced — essential for responsive chat UX. Structured output means constraining the model to return valid JSON matching a schema (via tool calling or response-format), so downstream code can rely on it. Prefer structured output over parsing free text.

B4 Embeddings, vectors & RAG

An **embedding** turns text into a vector; similar meanings sit close together. **RAG** (retrieval-augmented generation) embeds your documents, stores the vectors, and at query time retrieves the closest chunks to ground the model on your data. You don't need to build this from scratch on AWS — Bedrock Knowledge Bases do it — but you must understand chunking, top-k retrieval, and why bad retrieval (not the model) causes most

wrong answers.

Readiness

You can explain embeddings, cosine similarity, chunking and top-k, and why retrieval quality bounds RAG quality.

B5 Prompt engineering & evals basics

- Be explicit: role, task, constraints, output format. Show 1–2 examples for tricky formats.
- Put rules the model must not break in the system prompt; back-stop them with Guardrails/Policy (Guides 2–3) — prompts alone aren't security.
- **Evals** = a set of representative inputs with expected behaviour, scored automatically. They are how you catch regressions when you change a prompt, model, or tool. Start an eval set on day one.

Readiness

You can write a structured prompt and stand up a small eval set (inputs → expected behaviour).

Part C · AWS account & security from zero

Agents act on real systems with real permissions, so security *is* the foundation. This part takes a brand-new AWS footprint to a safe baseline. Do it once; reuse forever.

C1 Account bootstrap & root hygiene

- 1 Create the AWS account; immediately enable **MFA on the root user** and then stop using root for daily work.
- 2 Set up **AWS Organizations** if you'll have more than one account; create separate accounts (or at least separate environments) for **dev / staging / prod**.
- 3 Turn on a **budget + billing alarm** (e.g. alert at 50/80/100% of a monthly cap) before doing anything else.
- 4 Enable **CloudTrail** (audit log of every API call) and **AWS Config** for a baseline audit trail.
- 5 Pick a home **Region** (one that has Bedrock + AgentCore) and be consistent.

Pitfall

Building on the root user or one shared IAM user with AdministratorAccess. It fails every audit and one leaked key compromises everything. Use IAM Identity Center (below).

C2 IAM Identity Center, CLI & profiles

Use **IAM Identity Center** (formerly AWS SSO) for human access: short-lived credentials, no long-lived keys on laptops. Configure the CLI with SSO and named profiles.

```
aws configure sso          # set SSO start URL, region, account, role
aws sso login --profile dev # short-lived creds, refreshed on demand

# ~/.aws/config
[profile dev]
sso_session = corp
sso_account_id = 111122223333
sso_role_name = Developer
region = us-east-1

aws sts get-caller-identity --profile dev # confirm who you are
```

Readiness

You authenticate with SSO/short-lived creds, use named profiles per environment, and never store static keys on disk.

C3 IAM deep: policies, roles, least privilege

This is the concept you must be strongest on. Four building blocks:

Concept	What it is
Principal	Who/what is acting — a user, role, or service

Concept	What it is
Policy	JSON granting/denying actions on resources, optionally with conditions
Role	An identity a principal or service <i>assumes</i> to get temporary permissions
Trust policy	Who is allowed to assume a role (the role's 'who can wear this hat')

Agents and Lambdas run with an **execution role**. Grant it only the exact actions it needs — least privilege. Example: a tool that only invokes one Bedrock model and reads one S3 prefix.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["bedrock:InvokeModel"],
      "Resource": "arn:aws:bedrock:us-east-1::foundation-model/anthropic.*"
    },
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::bank-policies/*"
    }
  ]
}
```

- Prefer **roles** over users for anything programmatic; prefer **managed identities** over keys.
- Use **permission boundaries** and SCPs to cap what even admins can do in prod.
- Scope *Resource* ARNs tightly; avoid "*Resource*": "*" except for genuinely global read-only actions.
- Add **conditions** (source VPC, MFA, tag) for sensitive actions.

Readiness

You can read/write an IAM policy, explain assume-role + trust policy, and design a least-privilege execution role for one tool.

C4 Secrets, KMS, budgets & tagging

- **Secrets Manager** / **Parameter Store** for API keys, DB creds, tokens — never in code or env files committed to git. (AgentCore Identity vaults third-party creds for you; for everything else, use these.)
- **KMS** encrypts data at rest (S3, DynamoDB, logs). Use customer-managed keys for regulated data and control who can decrypt.
- **Budgets + Cost Explorer**: alarms on spend; tag every resource (*project, env, owner*) so cost is attributable.
- **Tagging** is also how you enforce attribute-based access control and clean up later.

Readiness

No secret lives in code; data at rest is KMS-encrypted; spend is alarmed and resources are tagged.

Part D · Core AWS services for agents

You don't need all of AWS — you need this handful well, because AgentCore and your tools sit on them.

D1 S3 · D2 Lambda · D3 API Gateway

S3 (object storage)

Buckets hold documents (for RAG), OpenAPI specs (for Gateway), artifacts, and logs. Know: bucket policies vs IAM, block-public-access (keep it on), versioning, server-side encryption, and presigned URLs for temporary access.

Lambda (serverless functions)

The workhorse for tools and glue. Know: the handler signature, environment variables, timeout/memory, the **execution role**, packaging (zip or container), and layers. Many agent tools are just a Lambda behind a clean schema.

```
def handler(event, context):
    account_id = event['account_id']
    # ... query core banking ...
    return {'account_id': account_id, 'balance': 1240.55, 'currency': 'USD'}
```

API Gateway

Exposes Lambdas/services as HTTP(S) endpoints with auth (IAM, Cognito, JWT), throttling and usage plans. When you expose an enterprise API as an agent tool, it's usually API Gateway + Lambda behind an OpenAPI spec.

D4 DynamoDB · D5 Step Functions

DynamoDB (key-value / document store)

Fast, serverless store for agent state, case records, idempotency keys, and conversation metadata. Know: partition key vs sort key, single-table basics, on-demand capacity, and TTL for expiring sessions. (AgentCore Memory abstracts much of this, but you'll still use DynamoDB for app state.)

Step Functions (workflow orchestration)

A managed state machine for **deterministic** multi-step processes with retries, parallelism, wait states and human-approval steps. Use it for the governed spine of a process and call an agent only for the steps that need judgement (you'll see this in Guide 2's ops example).

When to use which

Dynamic, model-decides-next-step → agent loop. Fixed, auditable, must-always-run-the-same-way → Step Functions. Real systems combine both.

D6 CloudWatch, X-Ray & OpenTelemetry

- **CloudWatch Logs** — where Lambda/Runtime logs land; learn Logs Insights queries.
- **CloudWatch Metrics + Alarms** — error rate, latency (p95/p99), invocation count, custom business metrics; alarm on them.
- **X-Ray** — distributed traces across services.
- **OpenTelemetry (OTEL)** — the open standard AgentCore Observability emits; learn the trace/span model because agent debugging is reading traces, not grepping logs.

Readiness

You can find a log, write an alarm on error rate + latency, and read a trace span tree.

D7 VPC basics · D8 ECR & Docker

VPC (private networking)

When agents/tools must reach private resources (a core-banking system, a private DB), they run in or connect to a VPC. Know: subnets (public/private), security groups, NAT, and **VPC endpoints** (reach AWS services like Bedrock/S3 privately without the public internet). For regulated data, keep traffic off the public net.

ECR & Docker

AgentCore Runtime runs your agent as a **container**, so you must be comfortable with a Dockerfile and pushing to **ECR** (Elastic Container Registry). The AgentCore toolkit can generate the Dockerfile, but you should understand what it builds.

```
# minimal agent container
FROM public.ecr.aws/docker/library/python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "my_agent.py"] # serves the runtime contract

# build + push to ECR
aws ecr get-login-password | docker login --username AWS --password-stdin <acct>.dkr.ecr.us-east-1.amazonaws.com
docker build -t my-agent . && docker tag my-agent <acct>.dkr.ecr.us-east-1.amazonaws.com/my-agent
docker push <acct>.dkr.ecr.us-east-1.amazonaws.com/my-agent:v1
```

Readiness

You can write a Dockerfile, build it, and push the image to ECR; you understand subnets, SGs and VPC endpoints at a basic level.

Part E · Bedrock setup & IaC

E1 Enable Bedrock & first Converse call

- 1 In the Bedrock console, open **Model access** and request/enable the models you need (e.g. Anthropic Claude). This is per-account, per-region.
- 2 Confirm your IAM principal has *bedrock:InvokeModel* (and *bedrock:Converse*) on those models.
- 3 Make the call with boto3:

```
import boto3
brt = boto3.client('bedrock-runtime', region_name='us-east-1')
resp = brt.converse(
    modelId='anthropic.claude-3-5-sonnet-20240620-v1:0',
    messages=[{'role':'user','content': [{'text':'Say hello as a bank assistant.'}]}],
    inferenceConfig={'maxTokens':200,'temperature':0.2})
print(resp['output']['message']['content'][0]['text'])
```

Pitfall

'AccessDenied' on InvokeModel almost always means model access isn't enabled in that region, or the IAM policy is missing the model ARN. Check both.

E2 Knowledge Bases & Guardrails

- **Knowledge Bases** — managed RAG: point at an S3 data source, pick an embedding model + vector store, and query it; great for grounding agents on policy docs, FAQs, contracts.
- **Guardrails** — configurable content filters, denied topics, word filters, and **PII detection/redaction** applied to model input and output. Essential for regulated domains; attach them to your agent calls.

E3 IaC: CloudFormation / CDK / Terraform

Click-ops doesn't scale and isn't auditable. Define infrastructure as code. Pick one and be consistent:

Tool	Use when
CloudFormation	Pure-AWS, declarative YAML/JSON; foundation under SAM and CDK
AWS CDK	You want real code (Python/TS) to generate CloudFormation; great DX
Terraform	Multi-cloud or you already standardise on it org-wide

```
# CDK (Python) sketch: a tool Lambda with a least-privilege role
fn = _lambda.Function(self, 'BalanceTool',
    runtime=_lambda.Runtime.PYTHON_3_12,
    handler='app.handler', code=_lambda.Code.from_asset('tool/'))
fn.add_to_role_policy(iam.PolicyStatement(
    actions=['dynamodb:GetItem'],
    resources=[table.table_arn]))
```

Readiness

You can stand up Bedrock access + a first Converse call, attach a Guardrail, and define a resource in IaC rather than the console.

Part F · Readiness checklist

Tick these before writing your first production agent. If any are shaky, revisit that section — Guides 2 and 3 assume all of them.

Python

- Isolated env + pinned deps; config from env, not code.
- Pydantic models for tool inputs, with constraints + descriptions.
- Comfortable with async/await and concurrent calls.
- Timeouts + backoff on every external call; structured logging; model-readable errors; pytest with mocks.

LLM

- Can reason about tokens/context/cost and pick temperature.
- Understand messages, system prompts, tool calling, streaming, structured output.
- Understand embeddings + RAG + why retrieval quality dominates.
- Can write a prompt and a small eval set.

AWS

- Account bootstrapped: root MFA, no daily root use, budgets, CloudTrail, SSO access.
- Fluent in IAM: policies, roles, assume-role, trust policy, least privilege; can write a scoped execution role.
- Secrets in Secrets Manager/Parameter Store; data KMS-encrypted; resources tagged.
- Working knowledge of S3, Lambda, API Gateway, DynamoDB, Step Functions.
- Can read CloudWatch logs/metrics, set alarms, read an OTEL/X-Ray trace.
- Basic VPC (subnets, SGs, endpoints); can build + push a Docker image to ECR.
- Bedrock model access enabled; first Converse call works; Guardrails understood; one IaC tool chosen.

You're ready when...

You can take a single tool from idea to a deployed, least-privilege, observable Lambda — and call a Bedrock model from code. Everything agentic is composing those skills. Next: **Guide 2** builds a full enterprise agent on this foundation.

Sources: AWS documentation — IAM, Bedrock, Lambda, Step Functions, CloudWatch, ECR best-practice guides (docs.aws.amazon.com). Verified May 2026.